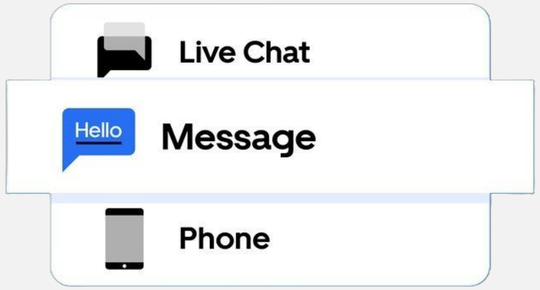
Innovation at Uber Eats:

Uber Eats AI Support Agent



Presented By : Jiaer Jiang, Samir Hendawi, Junyan Yue, Vicky Shen, Yuyan Zhang

Executive Summary

Uber Eats

1

Goal

Build an AI-powered conversational support agent for Uber Eats that automates customer service using Retrieval-Augmented Generation (RAG), LLM-driven intent classification, multi-turn actions, and safety guardrails.

2

Why It Matters

- **Reduces** manual support load
- Delivers **faster, consistent** responses grounded in real Uber Eats policies
- Demonstrates **production-ready** LLM + tool orchestration for real business workflows

3

What We Built

- Chat interface + tool panel for realistic customer interaction
- LLM as sole decision maker for intent classification (pure ReAct)
- RAG knowledge base of 60 Uber Eats help articles (scraped)
- 10 support tools: track, cancel, report missing/wrong/delivery, refund status, contact driver, ticket lookup, escalate

Outcomes

Our agent automates Uber Eats' most common support workflows using a RAG-first architecture. The LLM is the sole decision maker — it sees RAG context, tool descriptions, and conversation history, then chooses to answer or act. Code executes actions deterministically (tickets, refunds, order updates) while RAG retrieves real policies to inform customer-facing messages. Every response is scored by a real-time LLM-as-Judge evaluator with automatic retry on low quality.

Live Demo: https://uber-eats.streamlit.app/

Problem & Knowledge Base

Uber Eats

Why Uber Eats?

- **High volume & Low complexity**: tickets are simple like "where is my order?" "can I have a refund?" "items are missing"
- **Manual workflows are repetitive:** order lookup, policies checking, contacting drivers
- **User experience pain points**
  - Long wait time
  - Frustration on idling time when needed quick resolution
  - Requires both knowledge retrieval (refund policy, fees) AND action workflows (report, cancel, track)
- Clear **tool-use scenarios** showing agentic behavior
- Widely used in Toronto - relatable for demo

Knowledge Base & Data

60	articles scraped from help.uber.com
~180	chunks (400-char, 80-char overlap)
16	categories (orders, delivery, refunds...)
60	sample orders across 6 accounts
11	tools (5 instant, 4 ticket, 1 escalation)

Ingestion Pipeline: Articles scraped from *help.uber.com* → Chunking → Embedded via remote API (all-MiniLM-L6-v2) → Stored in ChromaDB → top-5 retrieval via cosine similarity

Source: help.uber.com/ubereats (publicly accessible)

Architecture & Design

Uber Eats

Input Guardrails	Mid-Tool Check	RAG Retrieval	LLM Decision	Execution	Evaluator
Regex injection + safety filter	Multi-turn param collection	ChromaDB top-5 chunks	Sole decision maker: answer or tool (JSON)	Tool call + RAG informed response	LLM-as-Judge score 1-5

Key Design:

The LLM is the sole decision maker — no pre-router, no regex routing. It sees RAG context + tool descriptions + conversation history and chooses to answer or call a tool. **Tools execute deterministically** (create ticket, issue refund, update DB). RAG retrieves relevant policy. LLM writes the customer message referencing that policy

Concepts Mapped

RAG Pipeline	400-char chunking, ChromaDB, remote embeddings	vectorstore.py	**Eval-Optimizer**	LLM-as-Judge, retry if < 3	agent.py
Augmented LLM	RAG + Tools + Memory (3 augmentations)	agent.py	**Guardrails**	Input (injection) + Output (length)	guardrails.py
ReAct Pattern	Retrieve > Reason > Act loop	agent.py	**Prompt Chaining**	Params > RAG > resolve > ticket	actions.py
Tool Use	10 tools via structured JSON	tools.py	**Memory**	SQLite sessions + state	memory.py

*See Appendix 1

Evaluation Results

Uber Eats

Real-Time Evaluator

LLM-as-Judge on every chat message

Scores 1-5, retries if < 3

Rule-Based Suite

15 automated test cases across 5 categories

Intent + keyword + source

Metrics Runner

Intent, tool, argument, E2E, latency

eval_metrics_run.py

Test Suite (15 Cases)

Category	N	What It Checks	Key Validation
RAG Retrieval	4	Knowledge answers from help articles	Source attribution, keyword accuracy
Tool Routing	4	Pre-router + LLM selects correct tool	Intent match, order ID extraction
Multi-Turn	2	Parameter collection across messages	Ticket creation, DB updates verified
Guardrails	3	Injection, unsafe content, out-of-scope	Rejection message, no system leakage
Error Handling	2	Invalid order IDs, nonsense input	Graceful degradation, helpful redirect

93.3 %
Intent Accuracy
Correct intent routing

85.7%
Tool Selection
Right tool chosen

93%
E2E Success Rate
Full pipeline pass

100%
Argument Correctness
Parameter DB checks · Argument consistency

5s Latency
Avg response time

*See Appendix 2, 3 (Success Metrics Results, Test Set Design)

Learnings & Limitations

Uber Eats

What Worked Well

- ✓ LLM & tools is a powerful combination
- ✓ Data updates in real time
- ✓ Guardrails
- ✓ Tools handle common requests without the need for intervention
- ✓ Multi- turn requests

Key Challenges

- ✕ Ambiguity and under-specified queries
- ✕ Multi-intent handling
- ✕ Parameter validation and safety

Known Limitations + Future work

- Always a possibility of hallucinations
- Mock data used, real uber user data is much more complex
- **Latency**: evaluator adds a second LLM call per message (~2-4s overhead), and higher token usage (cost)
- Rejected access to actual documents that will tell us how Uber Eats customer support are instructed

Uber Eats

Thank You

Any Questions?

Appendix 1

System components (high level)

Layer	What it is	Main files
Frontend	Streamlit UI: login, chat, orders/tickets/account views, custom CSS	app.py
Orchestration ("brain")	Single entry process_message: guardrails → RAG → routing → tools or answer → output checks → evaluator	agent.py
Tool / API layer	**Tool schema** (names, params, categories) vs **execution** (SQLite orders/tickets, mock "Uber" behavior)	tools.py, actions.py
Knowledge / RAG	Curated JSON KB → chunked → embedded → Chroma retrieval	data/knowledge_base.json, vectorstore.py, config.py
Memory & state	SQLite: chat history, session-scoped tool state (multi-turn param collection)	memory.py
Data stores	SQLite DBs for users, orders, tickets (+ Chroma on disk under writable dir)	users_db.py, orders_db.py, tickets_db.py, config.py
LLM engine	HTTP client to an **OpenAI-compatible** chat endpoint; JSON mode for decisions; plain chat for resolutions / retries	llm_client.py, env / defaults in config.py

Appendix 2

Evaluation Metrics

Metric	Result	Details
Intent accuracy	14 / 15 (93.3%)	In 14 cases the system's intent label matched the gold "user goal" substring patterns; 1 mismatch (MULTI-1: model ended with tool:track_order vs gold patterns favoring a delivery-problem / missing-order path).
Correct tool selection	6 / 7 (85.7%)	Among the 7 scenarios that required a specific tool, 6 ended with the correct tool:* in the final intent; 1 did not (MULTI-1).
Argument correctness	7 / 7 (100%)	Among scenarios with explicit order/ticket argument checks in the harness, all satisfied the DB consistency rules (orders exist / invalid-order handling / ticket IDs resolvable).
End-to-end success	14 / 15 (93.3%)	14 cases passed the full rubric (intent + tool when applicable + args when applicable + keyword checks + sources when required); 1 failed (MULTI-1, driven by intent/tool vs gold).
Latency (substitute measure)	See table below	Best available measure: wall-clock wrapper timing of process_message() on the final user turn per case (includes guardrails, RAG, decision LLM, tools, output guardrails, and evaluator per script note).

Appendix 3

Test Set Design

Stratum	Count	IDs	What is being tested
RAG / knowledge	4	RAG-1 … RAG-4	Refund timing, delivery fees, no-contact delivery, Uber One. Expect knowledge-style intent, non-empty retrieved sources, and keyword overlap with expected concepts.
Tool routing	4	TOOL-1 … TOOL-4	Single-turn actions with real order_ids from seeded data (user_samir orders, e.g. 100001–100005): track, refund status, missing items (+ follow-up line), delivery damage/spill.
Multi-turn	2	MULTI-1, MULTI-2	MULTI-1: missing order_id then user supplies 100005 (param collection + report_delivery_problem). MULTI-2: escalation to human; in eval_metrics_run.py this was collapsed to one rich message so issue_summary + urgency can be satisfied in one turn (metrics runner); evaluation.py still uses three separate messages.
Guardrails	3	GUARD-1 … GUARD-3	Prompt injection, out-of-domain (weather), unsafe/harmful request. Expect blocked or out_of_scope intents and refusal-style wording.
Error / edge	2	ERR-1, ERR-2	ERR-1: non-existent order 999999 (track still invoked, response should be safe/clarifying). ERR-2: nonsense string (should not crash; helpful or clarifying tone).